



Bot Telegram per la Gestione delle Prenotazioni di Lezioni

Anno Accademico 2023/2024

Studente:

Corso: Ingegneria Del Software

Mario Riccio (Matricola: 353143)

Docente: Alfredo Milani

16/07/2024

Abstract

Questo progetto presenta lo sviluppo di un bot Telegram progettato per semplificare e ottimizzare il processo di prenotazione delle lezioni universitarie. Il bot offre un'interfaccia user-friendly che consente agli studenti di visualizzare, prenotare e gestire le loro partecipazioni alle lezioni in modo efficiente e immediato.

Le principali funzionalità del bot includono:

1. Autenticazione degli utenti tramite credenziali universitarie
2. Visualizzazione dell'elenco delle lezioni disponibili con dettagli su orari, docenti e capienza delle aule
3. Sistema di prenotazione delle lezioni con controllo in tempo reale della disponibilità dei posti
4. Gestione delle prenotazioni effettuate

Il bot è stato sviluppato in Python; utilizza l'API di Telegram per la comunicazione utente-bot e un database MySQL per memorizzare i dati. L'overall dell'architettura progettata garantisce la scalabilità, sicurezza e facilità di manutenzione del sistema.

In generale, il progetto mira a migliorare l'esperienza organizzativa degli studenti riducendo al tempo stesso lo stress dell'amministrazione universitaria. Implementare il bot è un passo significativo verso la digitalizzazione e l'efficienza del sistema educativo formale, un'espansione sostanziale delle opportunità e accesso all'istruzione superiore.

Obiettivo del Progetto

L'obiettivo principale di questo progetto è sviluppare un bot Telegram efficace e user-friendly per la gestione delle prenotazioni delle lezioni universitarie. Questo sistema mira a modernizzare e semplificare il processo di prenotazione, rispondendo alle seguenti esigenze:

1. **Accessibilità:** Fornire agli studenti un accesso rapido e facile alle informazioni sulle lezioni e alle prenotazioni attraverso la piattaforma Telegram, ampiamente utilizzata e facilmente accessibile da dispositivi mobili e desktop.
2. **Efficienza:** Automatizzare il processo di prenotazione delle lezioni, riducendo il carico di lavoro amministrativo e minimizzando gli errori umani nella gestione delle iscrizioni.
3. **Trasparenza:** Offrire agli studenti una visione chiara e in tempo reale della disponibilità dei posti per ciascuna lezione, facilitando una migliore pianificazione delle attività didattiche.
4. **Flessibilità:** Permettere agli studenti di gestire le proprie prenotazioni in modo autonomo, con la possibilità di effettuare o cancellare le prenotazioni in qualsiasi momento, nel rispetto dei limiti stabiliti.
5. **Analisi dei Dati:** Fornire la possibilità di raccogliere e analizzare dati sull'utilizzo delle lezioni, che potrebbero essere utili per l'ottimizzazione della pianificazione dei corsi.

Attraverso il raggiungimento di questi obiettivi, il progetto mira a migliorare l'esperienza degli studenti nella gestione delle loro attività didattiche, promuovendo una maggiore partecipazione alle lezioni e un utilizzo più efficiente delle risorse universitarie.

Analisi dei Requisiti

1. Glossario dei principali termini:

- Utente: Studente universitario che utilizza il bot per prenotare le lezioni.
- Bot: Applicazione software su Telegram che gestisce le interazioni con gli utenti.
- Lezione: Sessione didattica programmata con un docente specifico, in un'aula e orario definiti.
- Prenotazione: Richiesta di partecipazione a una lezione specifica da parte di uno studente.
- Aula: Spazio fisico dove si tiene la lezione, con una capienza massima definita.

2. **Requisiti Funzionali:**

2.1 Autenticazione e Gestione Utenti:

- Il sistema deve permettere agli utenti di loggarsi utilizzando le proprie credenziali universitarie.
- Gli utenti devono poter effettuare il login e il logout.
- Il sistema deve verificare l'autenticità delle credenziali fornite.

2.2 Visualizzazione Lezioni:

- Gli utenti devono poter visualizzare l'elenco delle lezioni disponibili.
- Per ogni lezione, devono essere visualizzati: Id e nome del corso, docente, data, orario, aula e posti disponibili.

2.3 Prenotazione Lezioni:

- Gli utenti autenticati devono poter prenotare una lezione disponibile.
- Il sistema deve verificare la disponibilità dei posti prima di confermare la prenotazione.
- Gli utenti devono ricevere una conferma dopo aver effettuato una prenotazione.

2.4 Gestione Prenotazioni:

- Gli utenti devono poter visualizzare le proprie prenotazioni attive.
- Gli utenti devono poter cancellare una prenotazione esistente.

- Il sistema deve aggiornare automaticamente la disponibilità dei posti dopo ogni prenotazione o cancellazione.

3. Requisiti Non Funzionali:

3.1 Prestazioni:

- Il bot deve rispondere alle richieste degli utenti entro 3 secondi.

3.2 Usabilità:

- L'interfaccia del bot deve essere intuitiva e facile da usare anche per utenti non tecnici.
- I comandi del bot devono essere chiari e ben documentati.

3.3 Scalabilità:

- L'architettura del sistema deve permettere l'aggiunta di nuove funzionalità in futuro.
- Il database deve essere progettato per gestire un aumento significativo del numero di utenti e lezioni.

3.4 Manutenibilità:

- Il codice deve essere ben documentato

4. Vincoli:

- Il bot deve essere sviluppato utilizzando l'API ufficiale di Telegram.
- Il sistema deve integrarsi con il database esistente degli studenti dell'università.
- Il database è stato progettato utilizzando StarUML per la modellazione concettuale e implementato su XAMPP per la gestione e l'hosting

Architettura del Sistema

1. Bot di Telegram

Ruolo: Gestisce la comunicazione con gli utenti.

Funzioni:

- **Ricezione Comandi:** Il bot riceve comandi dagli utenti attraverso la chat di Telegram.
- **Elaborazione delle Richieste:** Il bot interpreta i comandi e determina le azioni appropriate.
- **Interazione con il Database:** Il bot si connette al database per recuperare o aggiornare le informazioni necessarie.
- **Risposta agli Utenti:** Il bot invia risposte agli utenti basate sui dati ottenuti e sulle azioni eseguite.

2. Database

Ruolo: Memorizza le informazioni sugli utenti, lezioni e prenotazioni.

Funzioni:

- **Memorizzazione Dati Utente:** Contiene le credenziali e i dettagli degli utenti.
- **Memorizzazione Dati Lezioni:** Contiene le informazioni sulle lezioni disponibili, inclusi orari e capacità.
- **Memorizzazione Prenotazioni:** Registra le prenotazioni effettuate dagli utenti.
- **Accesso e Modifica Dati:** Fornisce un'interfaccia per il bot per accedere e modificare i dati necessari.

3. Utenti

Ruolo: Studenti che interagiscono con il bot.

Funzioni:

- **Invio Comandi:** Gli utenti inviano comandi al bot per effettuare varie operazioni (login, logout, prenotazione, ecc.).

- **Ricezione Risposte:** Gli utenti ricevono risposte e notifiche dal bot in base ai comandi inviati.

Descrizione del Flusso di Interazione

1. Login dell'Utente

- L'utente invia il comando di login con email e password.
- Il bot verifica le credenziali nel database.
- Se le credenziali sono corrette, il bot salva lo stato di login e conferma l'accesso all'utente.

2. Logout dell'Utente

- L'utente invia il comando di logout.
- Il bot rimuove le informazioni di login e conferma il logout all'utente.

3. Visualizzazione delle Lezioni

- L'utente richiede l'elenco delle lezioni disponibili.
- Il bot recupera i dati delle lezioni dal database.
- Il bot invia l'elenco delle lezioni all'utente.

4. Prenotazione di una Lezione

- L'utente invia il comando per prenotare una lezione specificando l'ID della lezione e la propria matricola.
- Il bot verifica la disponibilità della lezione nel database.
- Se ci sono posti disponibili, il bot registra la prenotazione e conferma all'utente.

5. Annullamento di una Prenotazione

- L'utente invia il comando per annullare una prenotazione specificando l'ID della lezione e la propria matricola.
- Il bot verifica l'esistenza della prenotazione nel database.
- Il bot annulla la prenotazione e conferma l'azione all'utente.

6. Visualizzazione delle Prenotazioni

- L'utente richiede di visualizzare le proprie prenotazioni.
- Il bot recupera le prenotazioni dal database.
- Il bot invia l'elenco delle prenotazioni all'utente.

7. Richiesta di Aiuto

- L'utente invia il comando di aiuto.
- Il bot fornisce informazioni su come usare i vari comandi disponibili.

Comunicazione e Sicurezza

- **Comunicazione:** La comunicazione tra il bot e il database avviene attraverso connessioni sicure.
- **Gestione degli Errori:** Il bot gestisce eventuali errori di connessione e query attraverso blocchi try-except, assicurando che le risorse siano sempre rilasciate correttamente.
- **Sicurezza dei Dati:** Le credenziali degli utenti e le informazioni sensibili sono gestite in modo sicuro per prevenire accessi non autorizzati.

Diagramma di classe completo

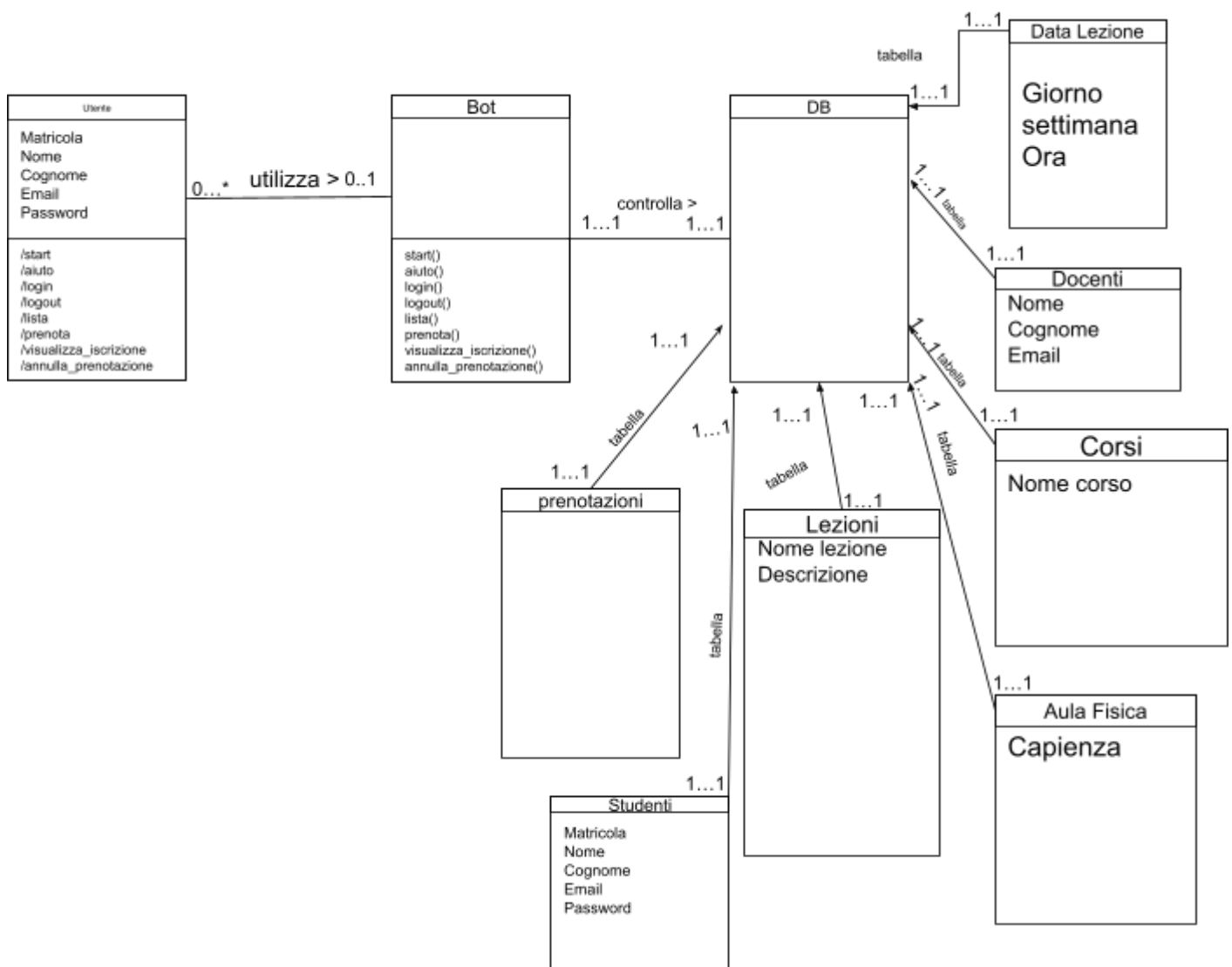
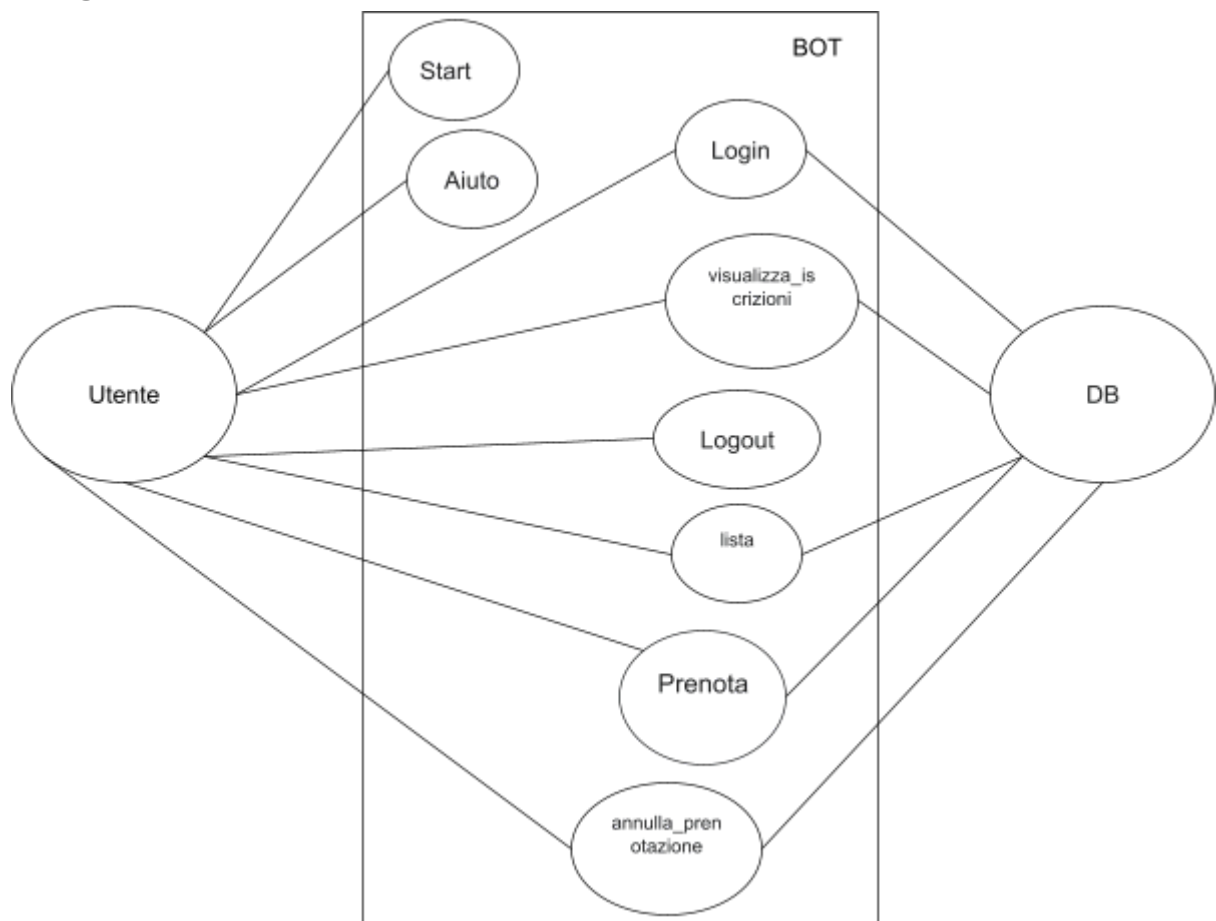


Diagramma di un caso d'uso



Descrizione Testuale

Start (/start)

- L'utente invia il comando **/start**.
- L'utente riceve un messaggio e inizia la conversazione.

Aiuto (/aiuto)

- L'utente invia il comando **/aiuto**.
- L'utente riceve un messaggio con l'elenco dei comandi.

Login (/login)

- L'utente invia il comando **/login** seguito da email e password.
- Il sistema verifica le credenziali e autentica l'utente.
- Se le credenziali sono corrette, l'utente viene loggato e riceve un messaggio di conferma.
- Se le credenziali sono errate, l'utente riceve un messaggio di errore.

Logout (/logout)

- L'utente invia il comando `/logout`.
- Il sistema effettua il logout dell'utente.
- L'utente riceve un messaggio di conferma del logout.

Visualizza lista delle lezioni (/lista)

- L'utente invia il comando `/lista`.
- Il sistema recupera la lista delle lezioni disponibili dal database.
- L'utente riceve un messaggio con l'elenco delle lezioni disponibili.

Prenota una lezione (/prenota)

- L'utente invia il comando `/prenota` seguito dall'ID della lezione e dalla propria matricola.
- Il sistema verifica la disponibilità della lezione e prenota un posto per l'utente.
- Se la prenotazione è riuscita, l'utente riceve un messaggio di conferma.
- Se la lezione è piena o se l'utente ha già prenotato, viene inviato un messaggio di errore.

Annulla una prenotazione (/annulla_prenotazione)

- L'utente invia il comando `/annulla_prenotazione` seguito dall'ID della lezione e dalla propria matricola.
- Il sistema verifica l'esistenza della prenotazione e la annulla.
- Se l'annullamento è riuscito, l'utente riceve un messaggio di conferma.
- Se non esiste una prenotazione per i dati forniti, viene inviato un messaggio di errore.

Visualizza iscrizioni (/visualizza_iscrizioni)

- L'utente invia il comando `/visualizza_iscrizioni`.
- Il sistema recupera la lista delle prenotazioni dell'utente dal database.
- L'utente riceve un messaggio con l'elenco delle proprie prenotazioni.

Diagramma di sequenza

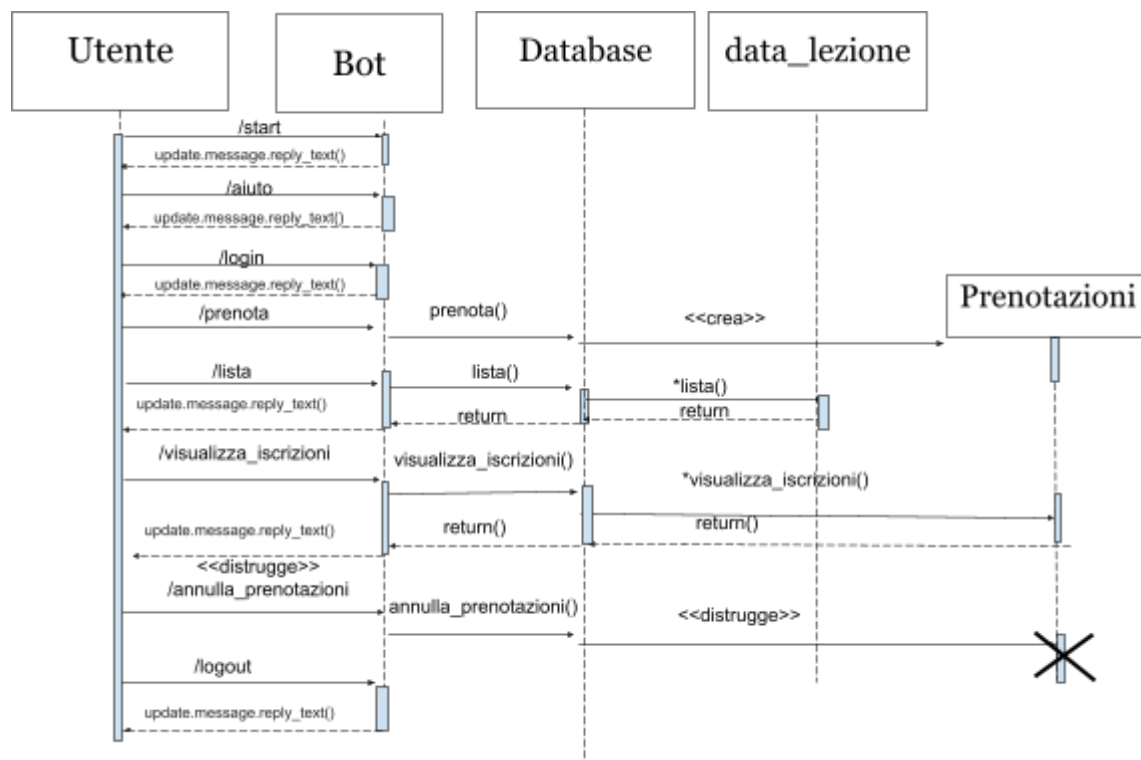


Diagramma di Collaborazione

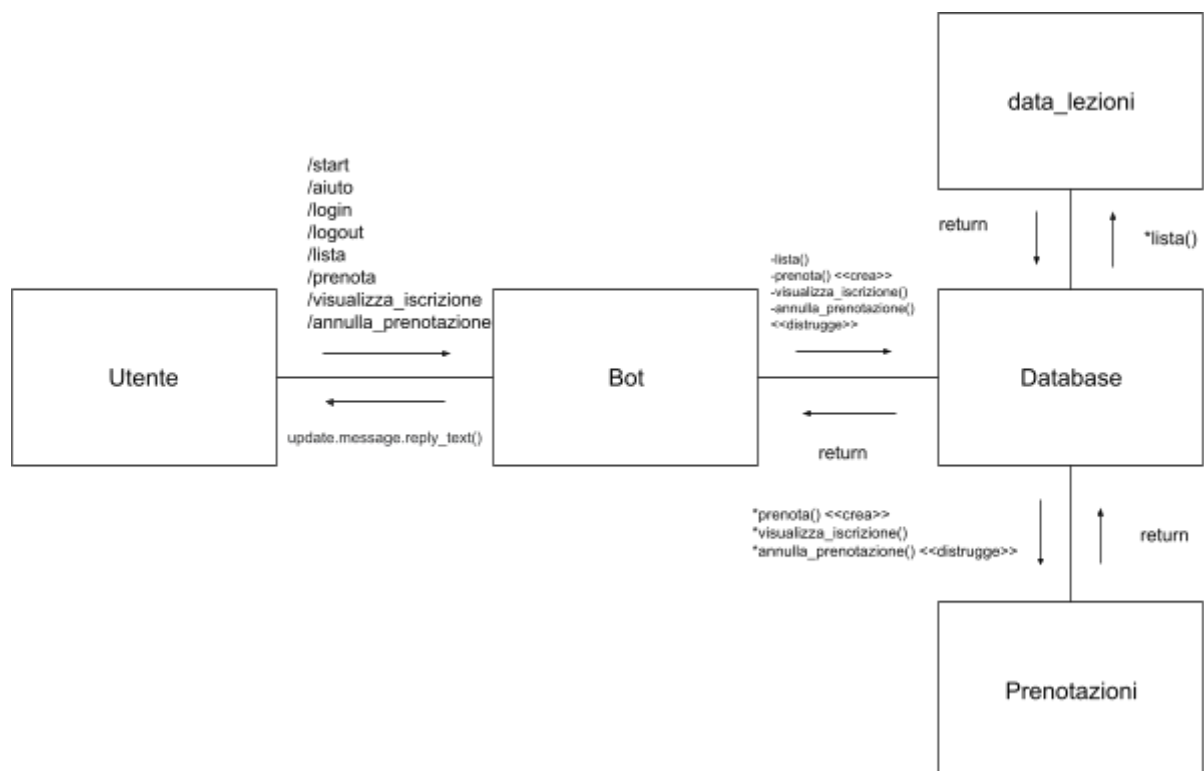


Diagramma di stato di un elemento del sistema

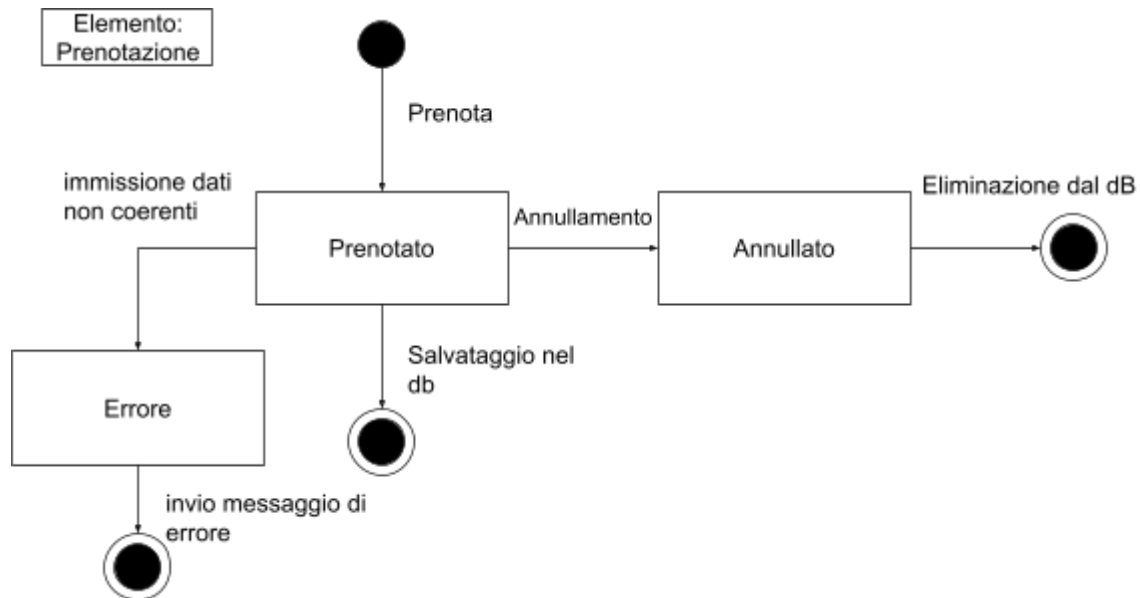
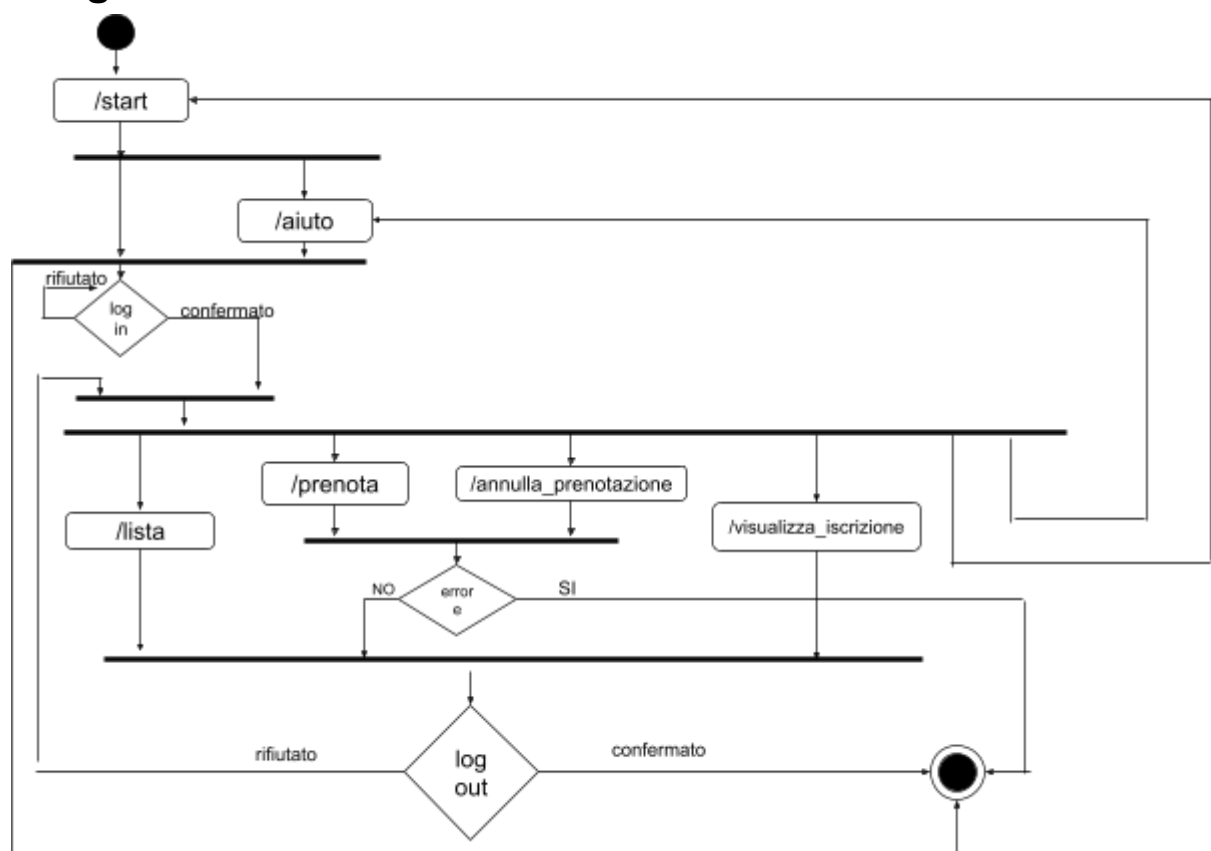


Diagramma di una attività



Realizzazione e Implementazione

Il bot Telegram per la gestione delle prenotazioni delle lezioni è stato implementato utilizzando Python e la libreria python-telegram-bot. Per la gestione del database, è stato utilizzato XAMPP, che include MySQL come sistema di gestione del database relazionale.

Ambiente di Sviluppo:

1. Linguaggio di programmazione: Python
2. Framework per il bot: python-telegram-bot
3. Database: MySQL (*tramite XAMPP*)

Struttura del Progetto:

Il progetto è composto da due componenti principali:

1. bot_telegram.py: File contenente tutto il codice Python per il funzionamento del bot
2. telebot.sql: File del database MySQL contenente la struttura
3. Dati_test.sql: Dati Test

Principali Funzionalità Implementate:

1. Autenticazione Utenti (/login, /logout)
2. Visualizzazione Lezioni (/lista)
3. Prenotazione Lezioni (/prenota)
4. Gestione prenotazioni (/visualizza_iscrizioni , /annulla_prenotazione)
5. Gestione Errori

Database: Il database MySQL, gestito tramite XAMPP, include le seguenti tabelle principali:

- studente
- corso
- lezioni
- data_lezione
- prenotazioni

- docenti
- aula_fisica

Configurazione del Database:

- Server: localhost (gestito da XAMPP)
- Username: root
- Password: [lasciata vuota per impostazione predefinita di XAMPP]
- Nome del database: telebot

Sicurezza:

- XAMPP è configurato per l'accesso locale, limitando l'esposizione del database

Implementazione Dettagliata:

Il file bot_telegram.py contiene tutte le funzioni necessarie per il funzionamento del bot, incluse:

- Connessione al database
- Gestione dei comandi Telegram (/start, /aiuto, /login, /logout, /lista, /prenota)
- Logica per la visualizzazione delle lezioni e la gestione delle prenotazioni

Il file telebot.sql contiene tutte le query SQL necessarie per:

- Creazione delle tabelle del database

Il file Dati_test.sql contiene tutte le query SQL necessarie per

- Inserimento dei dati iniziali

Test: I test dell'applicazione sono stati condotti utilizzando l'ambiente locale fornito da XAMPP per verificare il corretto funzionamento di tutte le funzionalità del bot.

Per i dettagli specifici del codice sorgente e la struttura del database, si prega di fare riferimento ai seguenti file aggiuntivi:

1. bot_telegram.py
2. telebot.sql

3. Dati_test.sql
4. requirements.txt

Test Funzionali:

Obiettivo: Verificare la robustezza e l'accuratezza del programma.

Comando `/login`

Test con credenziali corrette

- *Descrizione:* L'utente inserisce email e password corrette.
- *Input:* `/login email@example.com password123`
- *Output atteso:* Messaggio "Accesso effettuato come email@example.com." seguito dalla cancellazione del messaggio dell'utente
- *Controlli:* Verifica che l'email venga salvata in `context.chat_data['logged_in_user']`.

Test con credenziali errate

- *Descrizione:* L'utente inserisce email e/o password errate.
- *Input:* `/login email@example.com wrongpassword`
- *Output atteso:* Messaggio "Credenziali non valide." seguito dalla cancellazione del messaggio dell'utente

Test senza inserire credenziali

- *Descrizione:* L'utente non inserisce email e password.
- *Input:* `/login`
- *Output atteso:* Messaggio "Usa /login <email> <password>." seguito dalla cancellazione del messaggio dell'utente

Test con solo email

- *Descrizione:* L'utente inserisce solo l'email senza password.
- *Input:* `/login email@example.com`
- *Output atteso:* Messaggio "Usa /login <email> <password>."

Test con solo password

- *Descrizione:* L'utente inserisce solo la password senza email.
- *Input:* `/login password123`
- *Output atteso:* Messaggio "Usa /login <email> <password>."

Test con utente già loggato

- *Descrizione:* L'utente è già loggato e prova a fare di nuovo il login.
- *Precondizione:* `context.chat_data['logged_in_user']` è già settato.
- *Input:* `/login email@example.com password123`
- *Output atteso:* Messaggio " Sei già loggato come email@example.com." seguito dalla cancellazione del messaggio dell' utente.

Test con utente già loggato

- *Descrizione:* L'utente è già loggato e prova a fare di nuovo il login con credenziali diverse.
- *Precondizione:* `context.chat_data['logged_in_user']` è già settato.
- *Input:* `/login email2@example.com password1234`
- *Output atteso:* Messaggio " Devi fare il logout prima di accedere con un altro account. " seguito dalla cancellazione del messaggio dell' utente.

Comando `/logout`

Test con utente loggato

- *Descrizione:* L'utente è loggato e vuole effettuare il logout.
- *Precondizione:* `context.chat_data['logged_in_user']` è settato.
- *Input:* `/logout`
- *Output atteso:* Messaggio "Logout effettuato." e rimozione di `logged_in_user` da `context.chat_data`.

Test con utente non loggato

- *Descrizione:* L'utente non è loggato e prova a fare il logout.
- *Precondizione:* `context.chat_data['logged_in_user']` non è settato.
- *Input:* `/logout`

- *Output atteso*: Messaggio "Prima devi effettuare il login." senza errori.

Comando `/lista`

Connessione al database riuscita e lezioni presenti

- *Descrizione*: Verifica che il comando riesca a recuperare correttamente le lezioni disponibili quando la connessione al database è attiva e ci sono dati da visualizzare.
- *Setup*: Assicurati che la connessione al database sia attiva e che ci siano lezioni disponibili nel database.
- *Input*: Esegui il comando `/lista`.
- *Output atteso*: Messaggio con l'elenco delle lezioni disponibili.

Connessione al database riuscita ma nessuna lezione presente

- *Descrizione*: Verifica il comportamento del comando quando la connessione al database è attiva ma non ci sono lezioni disponibili.
- *Setup*: Assicurati che la connessione al database sia attiva e che non ci siano lezioni disponibili nel database.
- *Input*: Esegui il comando `/lista`.
- *Output atteso*: Messaggio indicando che non ci sono lezioni disponibili.

Errore durante l'esecuzione della query

- *Descrizione*: Verifica il comportamento del comando in caso di errore durante l'esecuzione della query per recuperare le lezioni dal database.
- *Setup*: Simula un errore durante l'esecuzione della query nel database.
- *Input*: Esegui il comando `/lista`.
- *Output atteso*: Messaggio di errore generico.

Comando `/prenota`

Prenotazione Standard

- *Descrizione*: Utente autenticato, lezione disponibile, posti liberi
- *Input*: Eseguire il comando `/prenota <id_lezione> <matricola>`
- *Output atteso*: Messaggio "Prenotazione effettuata con successo."

Prenotazione con Aula Piena

- *Descrizione:* Utente autenticato, lezione con tutti i posti occupati
- *Input:* Eseguire il comando `/prenota <id_lezione> <matricola>`
- *Output atteso:* Messaggio di errore: “La classe è piena, non è possibile prenotarsi per questa lezione.”.

Prenotazione Duplicata

- *Descrizione:* Utente autenticato, lezione già prenotata dallo stesso utente
- *Input:* Eseguire il comando `/prenota <id_lezione> <matricola>`
- *Output atteso:* Messaggio di errore: “Hai già una prenotazione per questa lezione.”

Prenotazione con Utente Non Autenticato

- *Descrizione:* Utente non autenticato
- *Input:* Eseguire il comando `/prenota <id_lezione> <matricola>`
- *Output atteso:* Messaggio di errore: “Effettua il login prima di poter prenotare.”

Prenotazione con ID Lezione Non Valido

- *Descrizione:* Utente autenticato, ID lezione inesistente
- *Input:* Eseguire il comando `/prenota <id_lezione> <matricola>`
- *Output atteso:* Messaggio di errore: “Lezione non trovata o non è possibile prenotare per questa lezione.”

Errore prenotazione

- *Descrizione:* Verificare il comportamento del comando in caso di errore durante l'accesso al database.
- *Setup:* Esegui il login con un utente valido.
- *Input:* Provoca un errore simulando un'eccezione durante l'esecuzione della query.
- *Output atteso:* Messaggio di errore generico.

Comando `/annulla_prenotazione`

Utente non autenticato

- *Descrizione:* Verificare che un utente non autenticato non possa annullare una prenotazione.
- *Input:* Eseguire il comando `/annulla_prenotazione <id_lezione> <matricola>` senza aver eseguito il login.
- *Output atteso:* Messaggio di errore: "Devi effettuare il login per annullare una prenotazione."

Input non valido

- *Descrizione:* Verificare che il comando gestisca correttamente input non validi.
- *Input:* Eseguire il comando `/annulla_prenotazione` senza argomenti o con argomenti non numerici.
- *Output atteso:* Messaggio di errore: "Uso: /annulla_prenotazione <id_lezione> <matricola>"

Matricola non corrispondente all'utente loggato

- *Descrizione:* Verificare che l'utente non possa annullare una prenotazione di un'altra matricola.
- *Setup:* Esegui il login con l'utente A (email di A).
- *Input:* Eseguire il comando `/annulla_prenotazione <id_lezione> <matricola_di_B>`.
- *Output atteso:* Messaggio di errore: "Non puoi annullare una prenotazione per questa matricola."

Prenotazione non esistente

- *Descrizione:* Verificare il comportamento del comando quando non esiste una prenotazione con i dati forniti.
- *Setup:* Esegui il login con un utente valido.
- *Input:* Esegui il comando `/annulla_prenotazione <id_lezione> <matricola>` dove non esiste una prenotazione.
- *Output atteso:* Messaggio di errore: "Non esiste una prenotazione con questi dati."

Prenotazione annullata con successo

- *Descrizione:* Verificare il comportamento del comando quando una prenotazione viene annullata con successo.
- *Setup:* Esegui il login con un utente valido e assicurati che esista una prenotazione per i dati forniti.
- *Input:* Esegui il comando `/annulla_prenotazione <id_lezione> <matricola>`.
- *Output atteso:* Messaggio di successo: "Prenotazione annullata con successo."

Errore annullamento delle prenotazioni

- *Descrizione:* Verificare il comportamento del comando in caso di errore durante l'annullamento delle prenotazioni dal database.
- *Setup:* Esegui il login con un utente valido.
- *Input:* Provoca un errore simulando un'eccezione durante l'esecuzione della query.
- *Output atteso:* Messaggio di errore generico.

Comando `/visualizza_iscrizioni`

Utente non autenticato

- *Descrizione:* Verificare che un utente non autenticato non possa visualizzare le prenotazioni.
- *Setup:* Assicurati che l'utente non abbia effettuato il login.
- *Input:* Esegui il comando `/visualizza_iscrizioni`.
- *Output atteso:* Messaggio di errore: "Devi effettuare il login per visualizzare le tue iscrizioni."

Utente autenticato senza prenotazioni

- *Descrizione:* Verificare che un utente autenticato senza prenotazioni visualizzi il messaggio corretto.
- *Setup:* Esegui il login con un utente che non ha prenotazioni nel database.
- *Input:* Esegui il comando `/visualizza_iscrizioni`.
- *Output atteso:* Messaggio: "Non hai prenotazioni al momento."

Utente autenticato con prenotazioni

- *Descrizione:* Verificare che un utente autenticato con prenotazioni visualizzi correttamente le sue prenotazioni.
- *Setup:* Esegui il login con un utente che ha almeno una prenotazione nel database.
- *Input:* Esegui il comando `/visualizza_iscrizioni`.
- *Output atteso:* Messaggio con la lista delle prenotazioni dell'utente.

Errore nel recupero delle prenotazioni

- *Descrizione:* Verificare il comportamento del comando in caso di errore durante il recupero delle prenotazioni dal database.
- *Setup:* Esegui il login con un utente valido.
- *Input:* Provoca un errore simulando un'eccezione durante l'esecuzione della query.
- *Output atteso:* Messaggio di errore generico.

Valutazione del Grado di Copertura dei Test

1. Login Utente

- **Test eseguiti:**
 - Test con credenziali corrette
 - Test con credenziali errate
 - Test senza inserire credenziali
 - Test con solo email
 - Test con solo password
 - Test con utente già loggato (stesso utente)
 - Test con utente già loggato (diverso utente)
- **Stato Copertura:** Completa
- **Motivazioni:** Tutti i possibili scenari di login sono stati testati, inclusi i casi di errore e le condizioni speciali come utente già loggato.

2. Logout Utente

- **Test eseguiti:**
 - Test con utente loggato
 - Test con utente non loggato
- **Stato Copertura:** Completa
- **Motivazioni:** Entrambi i casi principali sono stati testati per garantire che il logout funzioni correttamente in presenza e assenza di utenti loggati.

3. Visualizza Lista Lezioni

- **Test eseguiti:**
 - Connessione al database riuscita e lezioni presenti
 - Connessione al database riuscita ma nessuna lezione presente
 - Errore durante l'esecuzione della query
- **Stato Copertura:** Completa
- **Motivazioni:** Tutti gli scenari relativi al recupero delle lezioni sono stati testati, inclusi casi di successo e fallimento della query.

4. Prenotazione Lezione

- **Test eseguiti:**

- Prenotazione standard
- Prenotazione con aula piena
- Prenotazione duplicata
- Utente non autenticato
- ID Lezione non valido
- Errore durante l'esecuzione della query
- **Stato Copertura:** Completa
- **Motivazioni:** Tutti i possibili scenari di prenotazione sono stati testati, inclusi errori e condizioni speciali come aula piena o prenotazione già esistente.

5. Annulla Prenotazione

- **Test eseguiti:**
 - Utente non autenticato
 - Input non valido
 - Matricola non corrispondente all'utente loggato
 - Prenotazione non esistente
 - Prenotazione annullata con successo
 - Errore durante l'annullamento delle prenotazioni
- **Stato Copertura:** Completa
- **Motivazioni:** Tutti i casi di annullamento delle prenotazioni sono stati testati, incluso il comportamento in presenza di errori o input non validi.

6. Visualizza Iscrizioni

- **Test eseguiti:**
 - Utente non autenticato
 - Utente autenticato senza prenotazioni
 - Utente autenticato con prenotazioni
 - Errore nel recupero delle prenotazioni
- **Stato Copertura:** Completa
- **Motivazioni:** Tutti i casi di visualizzazione delle prenotazioni sono stati testati, coprendo scenari di successo e fallimento nel recupero delle informazioni.

7. Gestione Errori

- **Test eseguiti:**
 - Errori durante l'esecuzione delle query (varie situazioni)
 - Errore prenotazione
 - Errore durante l'accesso al database

- **Stato Copertura:** Completa
- **Motivazioni:** Diversi scenari di errore sono stati testati per garantire che il bot gestisca correttamente situazioni inaspettate durante l'interazione con il database o altre operazioni.